

We will describe how to solve the last subtask which was enough to achieve a full score.

Subtask 3:

The solution consists of three parts: building the graph (which is a tree), preprocessing the tree and answering the queries.

- For each reservoir we determine the next one (i.e. where the water flows into when the given reservoir overflows). To do this efficiently, we can go from bottom to top and maintain the stack of the reservoirs in ascending order of their diameters. Each time a new reservoir is processed we remove smaller or equal reservoirs from the top of the stack and insert the current reservoir, determining the next reservoir along the way. This can be done in $O(N)$. After that we connect each reservoir to the next one and we end up with a directed tree with a root in node 0 (that corresponds to the waterways).
- We use the method called “binary lifting”, most commonly used in LCA algorithm, where for each node we determine its 1st ancestor, 2nd ancestor, 4th ancestor, ..., 2^k -th ancestor. Additionally we precompute the total amount of water that the reservoirs up to the corresponding ancestor can hold. Since k is of the order of $\log N$, we will need $O(N \log N)$ time and $O(N \log N)$ memory for this part.
- For each query we keep moving to the farthest precomputed ancestor while the given amount of water doesn't exceed the sum of the capacities of the nodes traversed and find the answer. This step has $O(Q \log N)$ complexity.

The trivial solution, checking every diagonal was getting 10 points.

One way to improve it was to sort all the diagonals in ascending order of distance and query them in that order until a drawn one is found. This solution would get around 55 points.

In order to get 100 points it was enough to ask which diagonals are drawn from vertex 0 ($n-3$ questions in total). After that one could determine the answer: it is either one of the confirmed diagonals, or the one that connects the endpoints of two of the confirmed diagonals, those who are the closest to the opposite of vertex 0 from both sides.

To achieve a full score one could solve subtasks 2, 4 and 6 separately.

Subtask 2:

Go over the consecutive intervals of A with works that aren't better than B_1 and accumulate the lengths of those intervals that contain at least one work equal to B_1 . Since only one pass is needed it works in $O(N)$.

To solve other two subtasks we will need the following observations:

- The works in the final state will appear in the same relative order as they were originally
- student X may end up with student's Y 's work if and only if in the interval between X and Y (inclusive) there were no works that were better than X 's work

Any state that satisfies above requirements is reachable.

For each student X we can find the interval containing X and extending on both sides as much as possible that doesn't contain works better than X 's. This can be done in multiple ways:

- Simple seek in each direction until we find better work, in $O(N^2)$ which is good enough for most subtasks
- Using data structures (RMQ/Segment tree), in $O(N \log N)$
- Single pass in each direction maintaining sorted stack, in $O(N)$, similar to the one described in the task Fountain

Using these precomputed intervals we can answer whether student Y can end up with student X 's work in $O(1)$ and doing these checks at appropriate times will be needed in both subtasks.

Subtask 4:

We have to find the longest subsequence in B such that corresponding values in A are in non-decreasing order. This can be done by solving a well known Longest Increasing Subsequence (LIS) problem with $O(N \log N)$ solution.

Subtask 6:

We have to match the most values from B to A in the same order, which can be done in $O(N^2)$ using the standard 2D dynamic programming similar to the one that solves Longest Common Subsequence (LCS) problem.

Subtask 1 (25 points):

The main idea is to simulate bubble sort, where the array is sorted by swapping adjacent members of an array. We can perform a swap of two adjacent members of the array A_i and A_j by three-step operation (Where $\oplus$ is XOR):

1. $A_j \oplus A_i$
2. $A_i \oplus A_j$
3. $A_j \oplus A_i$

Array will transform like this: $[A_i, A_j] \rightarrow [A_i, A_j \oplus A_i] \rightarrow [A_j, A_j \oplus A_i] \rightarrow [A_j, A_i]$.

Since the bubble sort can take at most $\frac{n*(n-1)}{2}$ swaps to sort, our method will finish in at most $\frac{n*(n-1)}{2} * 3 \leq 40000$ operations.

Subtask 2 (35 points):

Here, since swapping is not a viable option, it's likely that we should rearrange the array with transformed values and then recover original values. Recovering the original values is important, because the resulting array should not have equal members and preserving original array elements is one way to ensure that. Our approach can be divided into n stages. In each stage, we will swap the largest element to the end of the unsorted part of the array. Consider the array $A = [a, b, c, d, e, f]$, where c is the maximal element. Firstly, we transform our array to the following one in $n - 1$ operations: $[a \oplus b, b \oplus c, c \oplus d, d \oplus e, e \oplus f, f]$, after that, we will make following operations: $[A_{c+1}, A_c], [A_{c+2}, A_{c+1}], \dots, [A_n, A_{n-1}]$, so the array will change to $[a \oplus b, b \oplus c, c \oplus d, c \oplus e, c \oplus f, c]$. After that, we can change left part of A same way: $[a \oplus c, b \oplus c, c \oplus d, c \oplus e, c \oplus f, c]$. Last two steps took maximum of $n - 1$ (when the maximum element is the first one) operations in total, i.e. each stage can take $2 * n - 2$ operations. As we can see, all the elements, apart from the last, are the original elements XOR-ed with c . Now we can move on to the next stage, but we have to pick the maximum from the **original** array, not the transformed one. For example, if the second maximum is e , we will move fourth element of A to fifth position (as c already holds the last) even if some element became bigger after the previous transformations.

Suppose that A would be increasingly sorted as follows: $[b, f, a, d, e, c]$. Then, after completing all the stages, which takes $n * (n - 1)$ operations, A will be like this: $[b \oplus f, f \oplus a, a \oplus d, d \oplus e, e \oplus c, c]$ (for example, like in the first stage, at the end of the second stage, last (fifth) element would be $e \oplus c$, while all the elements before fifth would be XOR-ed with $e \oplus c$ and $a \oplus c \oplus e \oplus c = a \oplus e$, so we would have a similar situation as at the end of the first stage). In the final sweep, we can recover original elements of A doing following operations: $[A_{n-1}, A_n], [A_{n-2}, A_{n-1}], \dots, [A_1, A_2]$. Finally, array is strictly increasingly sorted and we took total of $n^2 - 1$ operations, which just below the limit when $n = 200$.

Subtask 3 (40 points):

This subtask does not require distinct numbers, which gives us more freedom. We can sort any array A in $O(n \log n)$ operations in $\log n$ sweeps. In each sweep we find the leftmost element in A with the largest bit in its binary representation and then drag it to the end of the array. Suppose that such element is A_i . We can perform the following operations:

1. If A_{i+1} does not contain A_i 's maximum bit, then $A_{i+1} \oplus A_i$, after which A_{i+1} will contain that bit
2. $A_i \oplus A_{i+1}$ which removes maximal bit from A_i . In this way, we will drag this bit to the end in at most $2 * n$ operations.

It's easy to see that after each sweep the maximal bit we can find in unsorted part of A will decrease and since $A_i < 2^{20}$, after 20 sweeps only zeros will be left besides the elements to which we already dragged some maximal bits, which is non-decreasing, thus the array is sorted. In total, this method takes $2 * n * 20$ operations, which is exactly 40000 when $n = 1000$.

Subtask 1 (16 points):

In this subtask we can sort the cards in 51 swaps, so the second player can choose $target$ -th card and be sure that the guess is correct.

Subtask 2 (20 points):

Here, since we have S swaps and $N - S$ guesses, the first player can put first S cards on corresponding indexes and left all others in arbitrary places, if $target \leq S$, the second player will reveal $target$ -th card and guess first try. Otherwise in $N - S$ guesses the unsorted part can be fully searched.

Subtask 3 (22 points):

In this subtask we have $\frac{n}{4}$ swaps and $\frac{n}{2} + 1$ guesses. We can divide our deck in two predetermined parts (for example, odd and even indexes or first and second halves). For simplicity, we take odd and even indexes. Now we should observe the deck and see how many odd-valued cards are in even indexes (it's easy to see that this number will be equal to the number of even-valued cards on odd indexes). It's obvious, that either that number will be less or equal to $\frac{n}{4}$, or the number of even-valued cards on even indexes. So, in at most $\frac{n}{4}$ swaps, we can either put all the even-valued cards, or all the odd-valued cards on even indexes (after that, all the values of the odd indexed cards will have the same parity too).

On the first guess, the second player should check if odd indexes hold odd-valued cards by revealing the first card and after that, we have $\frac{n}{2}$ tries and $\frac{n}{2}$ possible locations of $target$, so the problem can be solved by full search in corresponding half.

Subtask 4 (18 points):

In this subtask we have 1 swap and $\frac{n}{2}$ guesses. If we interpret our deck as a graph, where i -th card is connected to the v_i -th one (where v_i is the value of the i -th card),

Card Trick

we will get a graph, where every card is a part of exactly one simple cycle. If we follow the cycle of card t by first revealing $target$ -th card, then v_t -th, v_{v_t} -th and so on, we will find the card $target$ in exactly s_t guesses (where s_t is the size of that cycle), because the last card of the cycle should point to index $target$, which means it has a value of $target$.

The main issue of this solution is that we can have cycles larger than $\frac{n}{2}$, but we can't have more than one such cycle for obvious reasons. If such cycle exists, we can swap $\frac{n}{2} + 1$ -th and the first element of cycle (you can enumerate the elements of the cycle any way you want as long as i -th card points to $i + 1$ -th and the last one points to the first). After the swap, $\frac{n}{2}$ -th card will still point to the index of $\frac{n}{2} + 1$ -th card, but now that card points to the second one, so now this means that the size of one of newly formed cycle is exactly $\frac{n}{2}$, while the size of remaining cycle is $s_i - \frac{n}{2} \leq \frac{n}{2}$, so now the second player can definitely find the desired card in time.

Subtask 5 (24 points):

This subtask is just a generalization of the last one. If we have S swaps and T guesses, on each swap we can take one of the cycles with size greater than T (if one exists) and cut the T -sized cycle out of it. Algorithm of the second player remains exactly the same.

Subtask 1 (20 points):

Since there is only one component the second player is going to claim it. It was enough to calculate the size of that component which can be done by counting the number of already claimed boxes and subtracting from the total number.

Subtask 2 (20 points):

The number of positions players could get (which is $2^{\text{\#unjoined sides}}$) was small enough to try and determine the optimal strategy from each possible state using dynamic programming or memoization.

Subtask 3 (20 points):

The general strategy should be about claiming the boxes whenever possible but it is important to understand that sometimes when you have the opportunity to take a whole chain of length n it is better to claim $n-2$ boxes and leave the last 2 to the other player by making a move that doesn't complete any of those 2 boxes. Using this you are forcing the opponent to make the bad move later that will give the current player an even bigger component back. With this in mind, one could find the components (whose amount is at most 2) and find the answer using simple casework.

Subtask 4 (20 points):

Due to the constraints each component will be either a chain or a loop. The state of the game can be described as the lengths of the remaining chains and loops. There won't be many components in this subtask and one could solve it by solving each state, similar to subtask 2, this time the number of states being $O(2^{\text{\#of components}})$.

Subtask 5 (20 points):

Notice that if one player has to give up the component of a certain kind (chain or loop), it is better to give up the smallest one. Using this the number of states that optimal solutions can reach can be reduced significantly to some polynomial amount and this time dynamic programming/memoization approach can be used to achieve the full score.